

INFORME TÉCNICO

Sistema "Perlitas de Eucarion"

Desarrollo FullStack II_008D

Integrantes: Yuliana Calvucura, Francisco Riveras y Catalina Salinas
Fecha: 10-09-2026

Índice

1. Introducción
 2. Objetivos del Documento
 3. Stack Tecnológico del Frontend
 4. Arquitectura y Estructura de Archivos
 5. Detalle de las Páginas HTML
 - 5.1 Index.html — Página de inicio
 - 5.2 Registrar.html — Registro de ventas
 - 5.3 Ventas.html — Listado de ventas
 - 5.4 Productos.html — Gestión de productos
 - 5.5 Materiales.html — Gestión de materiales
 6. Hoja de Estilos (Estilo.css)
 7. Validación de Formularios en JavaScript
 8. Elementos Multimedia e Interactividad
 9. Buenas Prácticas y Uso de HTML5 Semántico
 10. Conclusión y Oportunidades de Mejora
- Anexos

1. Introducción

Este informe técnico documenta la programación del frontend del sistema: "Perlitas de Eucarion", una aplicación web desarrollada en HTML, CSS y JavaScript orientada a digitalizar la gestión de una pyme de bisutería y productos artesanales.

A diferencia de una arquitectura de backend con microservicios, este repositorio contiene un sitio web con varias paginas construido con HTML plano, una hoja de estilos CSS compartida y scripts JavaScript embebidos en cada página, encargados de la validación de formularios en el lado del cliente. Este documento describe la estructura del proyecto, el detalle técnico de cada archivo HTML, el diseño visual definido en la hoja de estilos y la lógica de validación implementada en JavaScript.

2. Objetivos del Documento

- Describir la estructura general del sitio web y la función de cada archivo HTML.
- Documentar los elementos y etiquetas HTML5 utilizados para construir la interfaz.
- Explicar el diseño visual aplicado mediante la hoja de estilos Estilo.css.
- Detallar la lógica de validación de formularios implementada en JavaScript.
- Identificar buenas prácticas aplicadas y oportunidades de mejora en el código del frontend.

3. Stack Tecnológico del Frontend

No se emplean frameworks de frontend (React, Vue, Angular) ni preprocesadores CSS; toda la interfaz se construye con HTML, CSS y JavaScript puros, lo que la hace liviana y fácil de ejecutar abriendo directamente los archivos en un navegador, sin requerir un servidor de aplicaciones.

4. Arquitectura y Estructura de Archivos

El repositorio se organiza como un sitio estático de cinco páginas HTML que comparten una única hoja de estilos y una estructura de navegación común (header + nav + main + footer). No existe backend ni persistencia real de datos: los formularios validan la información en el cliente mediante JavaScript, y las tablas de "Productos", "Materiales" y "Ventas" muestran datos de ejemplo escritos directamente en el HTML (hardcodeados), a la espera de una futura integración con una API o base de datos.

Las cinco páginas replican la misma barra de navegación (Inicio, Registrar venta, Ver ventas, Productos, Materiales), lo que otorga consistencia visual y de navegación al sitio, aunque implica que cualquier cambio en el menú deba repetirse manualmente en cada archivo, al no existir un componente de navegación reutilizable ni el uso de un framework o de includes del lado del servidor.

5. Detalle de las Páginas HTML

5.1 Index.html — Página de inicio

Es la página de entrada del sistema. Declara el documento HTML5 estándar (<!DOCTYPE html>, meta charset UTF-8, meta viewport para diseño responsivo) y enlaza la hoja de estilos externa. Su cuerpo se organiza con etiquetas semánticas: header (título y logo del emprendimiento), nav (menú de navegación) y main con dos secciones: una de bienvenida ("Gestión del emprendimiento") y otra con enlaces directos a las demás páginas ("Opciones").

El aspecto más particular de esta página es un objeto JavaScript literal llamado `tema_musical`, que encapsula el título, la ruta del archivo de audio y dos métodos, `reproducir()` y `detener()`, que crean y controlan dinámicamente un objeto `Audio` del navegador. Estos métodos se invocan desde dos botones mediante el atributo `onclick`, permitiendo reproducir o pausar una pista de música de fondo sin recargar la página.

```
let tema_musical = { titulo: "...", archivo: "assets/SomeCatalanMysteries.mp3", audio: false,
  reproducir: function () { if (!this.audio) { this.audio = new Audio(this.archivo); } this.audio.play(); },
  detener: function () { if (this.audio) { this.audio.pause(); } } };
```

5.2 Registrar.html — Registro de ventas

Contiene un formulario (`form_venta`) con campos de texto para producto, cliente, método de pago, cantidad y total. Cada campo posee un `span` asociado con la clase `mensaje_error`, destinado a mostrar mensajes de validación. El botón "Registrar venta" no es de tipo `submit`, sino `type="button"` con un manejador `onclick="procesar()"`, por lo que el envío del formulario se controla completamente por JavaScript en lugar de depender del comportamiento nativo de envío HTML.

La función `procesar()` obtiene los valores ingresados mediante el objeto `FormData`, limpia mensajes de error previos y aplica dos funciones de validación reutilizables: `validar_texto()` (verifica que el campo no esté vacío tras eliminar espacios con `trim()`) y `validar_cantidad()` (verifica que el valor sea numérico, mediante `isNaN()`, y mayor a cero mediante `parseInt()`). Si un campo no es válido, se inserta el mensaje de error correspondiente en el `span` asociado usando `innerHTML`.

5.3 Ventas.html — Listado de ventas

Página de solo lectura que reutiliza el mismo header, nav y footer que el resto del sitio, y presenta en una tabla HTML (`table`, `caption`, `tr`, `th`, `td`) el listado de ventas registradas, con columnas Cliente, Producto, Cantidad, Método de pago y Total. Los datos mostrados están escritos directamente en el HTML como ejemplo, sin conexión a un backend ni a `Registrar.html`, por lo que una venta registrada en el formulario no se refleja aún en esta tabla.

5.4 Productos.html — Gestión de productos

Combina un formulario de registro de productos (nombre, categoría, precio y stock) con la misma lógica de validación en JavaScript utilizada en `Registrar.html` (funciones `validar_texto`, `validar_cantidad` y `limpiar_errores`, con nombres de función adaptados como `procesar_producto()`), junto con una tabla que muestra el catálogo de productos disponibles (Collar de perlas, Pulsera de perlas, Aros de titanio), también con datos de ejemplo fijos en el HTML.

5.5 Materiales.html — Gestión de materiales

Estructura idéntica a Productos.html pero orientada al inventario de insumos: formulario con nombre, cantidad, proveedor y costo unitario (función `procesar_material()`), y una tabla con el listado de materiales disponibles (perlas de vidrio, hilo encerado, cierres metálicos). Repite el mismo patrón de validación en JavaScript que las páginas de Registrar y Productos, evidenciando un patrón de código consistente aunque duplicado entre archivos.

6. Hoja de Estilos (Estilo.css)

El diseño visual del sitio se define íntegramente en un único archivo CSS externo, enlazado desde las cinco páginas mediante `<link rel="stylesheet">`. Se utiliza una paleta de colores en tonos rosado y vino, coherente con la identidad de marca de un emprendimiento de bisutería:

Elemento	Estilo aplicado
body	Fuente Helvetica/sans-serif, fondo rosado muy claro (#FFF5F7), texto vino (#602437)
header / footer	Fondo vino (#8A2846), texto blanco, padding de 15-20px
nav	Fondo rosado (#FFCAD4), lista horizontal (display: flex) sin viñetas
section	Tarjetas blancas con borde rosado y separación entre secciones
.mensaje_error	Texto en rosado fuerte (#B9375E) para resaltar errores de validación
.logo	Imagen posicionada de forma absoluta sobre el header (esquina superior derecha)

El menú de navegación se construye con `nav > ul > li` mostrados en fila mediante `display: flex`, y los enlaces cambian de color al pasar el cursor (`:hover`), aportando retroalimentación visual básica sin necesidad de JavaScript. No se utilizan media queries, por lo que el sitio no está optimizado de forma explícita para dispositivos móviles más allá del meta viewport declarado en el HTML.

7. Validación de Formularios en JavaScript

Las tres páginas con formularios (Registrar.html, Productos.html y Materiales.html) comparten el mismo patrón de validación del lado del cliente, compuesto por tres funciones que se repiten con ligeras variaciones de nombre:

- `validar_texto(texto)`: recorta espacios con `trim()` y confirma que el resultado tenga una longitud mayor a cero.
- `validar_cantidad(valor)`: recorta espacios, confirma que no esté vacío, que no sea NaN y que, convertido con `parseInt()`, sea mayor a cero.
- `limpiar_errores()`: recorre todos los elementos con la clase `mensaje_error` mediante `getElementsByClassName()` y vacía su contenido antes de cada validación.

La función principal de cada página (`procesar()`, `procesar_producto()` o `procesar_material()`) se dispara con el evento `onclick` del botón de envío —no con el evento `submit` del formulario—, obtiene los datos con el objeto `FormData`, y por cada campo inválido escribe el mensaje de error correspondiente en el `span` asociado mediante `innerHTML`. Cabe destacar que, si todos los campos son válidos, actualmente no se ejecuta ninguna acción visible (no

hay confirmación de éxito ni se agrega la fila a la tabla), ya que no existe conexión con un backend ni actualización dinámica del DOM tras un registro exitoso.

8. Elementos Multimedia e Interactividad

Index.html incorpora un elemento adicional de interactividad mediante el objeto de audio HTML5 (`new Audio()`) controlado por JavaScript, permitiendo reproducir y pausar una pista musical de fondo sin utilizar la etiqueta nativa `<audio>` visible en la interfaz. Además, la imagen del logo (`assets/image.jpg`) se muestra mediante una etiqueta `<img>` posicionada sobre el header con CSS. Estos recursos se almacenan en la carpeta `assets/`, separando el contenido multimedia del código de marcado.

9. Buenas Prácticas y Uso de HTML5 Semántico

- Uso consistente de etiquetas semánticas HTML5 (`header`, `nav`, `main`, `section`, `footer`) en lugar de `div` genéricos, lo que mejora la legibilidad y la accesibilidad del marcado.
- Formularios estructurados con `fieldset` y `legend`, y asociación explícita entre `label` y `input` mediante los atributos `for` e `id`.
- Separación de responsabilidades entre estructura (HTML), presentación (CSS externo) y comportamiento (JavaScript), aunque el JavaScript se mantiene embebido dentro de cada página en lugar de externalizarse en archivos `.js` independientes.
- Repetición del bloque de header, nav y footer, así como de las funciones de validación, en cada archivo HTML — código duplicado que podría reducirse con componentes reutilizables o con un archivo JavaScript común.
- Ausencia de atributos de accesibilidad adicionales (por ejemplo, `aria-live` en los mensajes de error) y de media queries para diseño responsivo.
- Los datos de las tablas (productos, materiales, ventas) están escritos directamente en el HTML, por lo que el sitio actúa como un prototipo de interfaz sin persistencia real de datos.

10. Conclusión y Oportunidades de Mejora

El repositorio analizado corresponde a un prototipo funcional del frontend de "Perlitas de Eucarion", construido enteramente con HTML5, CSS3 y JavaScript nativo. Cumple con el objetivo de presentar una interfaz de navegación consistente entre cinco páginas, formularios con validación básica en el cliente y una identidad visual propia definida mediante CSS. La lógica de validación es clara y sigue un patrón uniforme (`trim`, comprobación numérica y limpieza de errores) fácil de mantener y de extender.

Como próximos pasos se identifican las siguientes oportunidades de mejora:

- Extraer las funciones JavaScript comunes (validar texto, validar cantidad, limpiar errores) a un archivo externo compartido (por ejemplo, `scripts.js`) para eliminar la duplicación de código entre páginas.
- Conectar los formularios y las tablas con un backend real (API REST), reemplazando los datos estáticos de las tablas por información persistida en una base de datos.
- Agregar retroalimentación de éxito al registrar una venta, producto o material (mensaje de confirmación o actualización dinámica de la tabla vía DOM), no solo mensajes de error.

- Incorporar media queries u otra estrategia de diseño responsivo para mejorar la experiencia en dispositivos móviles.
- Modularizar el header, nav y footer para evitar repetir el mismo bloque de HTML en cada archivo.
- Adaptabilidad del código para una correcta y oportuna visualización de vista móvil.

Anexos

Repositorio del proyecto (frontend HTML):

<https://github.com/MisuWasTaken/Perlitas-de-Eucarion.git>

El repositorio de momento contiene cinco archivos HTML (Index, Registrar, Ventas, Productos y Materiales), una hoja de estilos (Estilo.css) y una carpeta assets con el logo del emprendimiento y una pista de audio utilizada como música de fondo en la página de inicio.